Tema IV: Generación de Código Objeto

La última fase que atraviesa un programa en su proceso de compilación es la **generación del código objeto**. Su objetivo principal es realizar la traducción del programa representado en 3 direcciones a una representación de bajo nivel, la cual puede ser directamente una representación en lenguaje máquina (código objeto) o una equivalencia en lenguaje ensamblador.

Para poder llevar a cabo esta última traducción es necesario tener en cuenta varios aspectos relativos a la máquina destino, es decir, al equipo en el cual se ejecutará finalmente el programa.

Entre los principales aspectos que se tienen que considerar de ese dispositivo de procesamiento están:

- La arquitectura del dispositivo.
- La naturaleza del conjunto de instrucciones soportada por ese dispositivo.
- El manejo de memoria y registros, entre otros.

Para el caso de nuestro proyecto escolar realizaremos la traducción de los cuádruplos a una representación equivalente en lenguaje ensamblador, por lo cual resulta indispensable recordar la estructura básica que guarda un programa en lenguaje ensamblador, así como los elementos indispensables de todos los programas en ensamblador.

Estructura General de un Programa en Lenguaje Ensamblador

```
.MODEL SMALL          ; (Directiva) Define el modelo de memoria
.STACK                ; (Segmento) Reserva espacio para la pila
.DATA                 ; Segmento de datos
  NUM1 DB 5           ; Variables
  NUM2 DB 3
  RESULTADO DB ?
.CODE                 ; Segmento de código
MAIN PROC             ; Procedimiento principal
  MOV AX, @DATA
  MOV DS, AX
  MOV AL, NUM1
  ADD AL, NUM2
  MOV RESULTADO, AL
  MOV AH, 4CH
  INT 21H
MAIN ENDP
END MAIN              ; Indica el punto de entrada del programa
```

Aspectos de la Generación de Código

Cuando se trata de la última traducción del programa a su versión final ejecutable, hay varios aspectos que debemos tomar en cuenta y que analizaremos uno a uno a continuación:

1. Entrada al Generador de Código

Lo primero que se debe tomar en cuenta es que el generador de código recibe como entrada la representación intermedia del programa fuente junto con la **tabla de símbolos**, en la cual está reunida la información de los identificadores, tales como:

- El nombre de la variable.
- El tipo de dato asociado a cada variable.
- De ser necesario, el valor inicial de las mismas.

> **Nota:** Es importante remarcar que el código que ingresa a la fase de generación del código objeto debe estar completamente fuera de errores y, de ser posible, optimizado.

2. Programa Objeto

- Aunque suena lógico, es importante analizar como un aspecto de diseño la salida que el generador de código produce, ya que todos los generadores de código producen una versión de bajo nivel del programa fuente y pueden tomar algunas de las siguientes formas:

a) Lenguaje Ensamblador: Esta forma de representación facilita la generación de código ya que utiliza instrucciones simbólicas y las macros de ensamblador que ayudan a generar código. Su desventaja es que esta forma de representación requiere un paso adicional para dejarlo en su versión totalmente ejecutable.

b) Lenguaje Máquina Absoluto: Cuando se traduce un programa al lenguaje máquina absoluto, implica que estamos traduciendo a un verdadero código

objeto, esto es, ceros y unos (0 y 1) que representan la secuencia del programa. El calificador *absoluto* implica que las direcciones señaladas en el código son las verdaderas direcciones físicas en las cuales se encuentra cada elemento.

- **Ventaja:** El programa ya está en una versión 100\% ejecutable y no requiere de procesos adicionales para la traducción de código o el cálculo de direcciones, haciendo que los programas se ejecuten más rápido.
- **Desventaja:** Si las direcciones señaladas en el código se encuentran ocupadas, el programa no puede ejecutarse.

c) Lenguaje Máquina Relocalizable: Son una variación del caso anterior en el que las direcciones utilizadas son un desplazamiento del equivalente real en donde se encuentra localizado cada elemento.

- **Ventaja:** Permite que el programa se ejecute en una localidad distinta de memoria cada vez que se ejecute.
- **Desventaja:** Introduce la necesidad de recalcular direcciones en cada ocasión.

Administración de memoria

La correspondencia entre los nombres (identificadores, variables, nombres de procedimientos y métodos) presentes en el programa fuente con las direcciones de objetos de datos en la memoria durante la ejecución se realiza en una colaboración entre la etapa inicial de compilación (análisis léxico) y la etapa final, generación de código objeto.

Recordemos que, en el análisis léxico, identificamos las palabras o cadenas que podían clasificarse como variables o identificadores, estas fueron registradas en la tabla de símbolos y posteriormente se les asoció un tipo de dato, y posteriormente esa información será utilizada para indicarle al cargador del programa cuánto espacio de memoria se requiere para cada variable, dependiendo de su tipo y que etiqueta tendrá el espacio de memoria específico.

Selección de instrucciones

En este aspecto del diseño de un generador de código, se va a elegir el conjunto de instrucciones que sea adecuado para traducir los cuádruplos a su representación de bajo nivel, esto depende **totalmente** del dispositivo final en el cual se ejecutará el programa. En términos generales existen dos grandes categorías de conjuntos de instrucciones, estas son:

- RISC
- CISC

Preguntas sobre RISC y CISC:

¿Qué significan las siglas?

- RISC: Reduced Instruction Set Computer (Computadora con Conjunto de Instrucciones Reducido).
- CISC: Complex Instruction Set Computer (Computadora con Conjunto de Instrucciones Complejo).

- ¿Para qué tipo de arquitecturas o usos es más adecuada?
  - RISC: Es idónea para dispositivos móviles, sistemas embebidos y arquitecturas donde se prioriza la eficiencia energética, menor consumo de batería y operaciones simples/rápidas (ej. smartphones, IoT, microcontroladores).
  - CISC: Es ideal para computadoras de escritorio, servidores y sistemas donde se requiere realizar tareas complejas directamente por hardware y se cuenta con alimentación eléctrica continua (ej. PCs de alto rendimiento y servidores corporativos).
- ¿Ventajas y desventajas clave?
  - RISC:
    - Ventajas: Ejecución más rápida por ciclo de reloj, menor consumo de energía, hardware más simple de diseñar.
    - Desventajas: Requiere código binario más largo (más instrucciones) para realizar tareas complejas, lo que exige compiladores más inteligentes.

- CISC:
  - Ventajas: Programas/código máquina más compactos (menos instrucciones por tarea), facilita el trabajo del compilador.
  - Desventajas: Hardware y decodificación de instrucciones más compleja, mayor consumo de energía y disipación de calor.

- Ejemplos reales de quién las usa actualmente:
  - RISC: Arquitecturas ARM (presentes en procesadores Snapdragon de Qualcomm, Apple Silicon M1/M2/M3/M4, iPhones) y RISC-V.
  - CISC: Arquitectura x86 / x86-64 (utilizada por procesadores Intel Core/ Xeon y AMD Ryzen/EPYC).

Asignación de registros

En la construcción del programa objeto es necesario tener en cuenta que se requiere colocar en registros algunasvariables para poder ejecutar las operaciones. Este aspecto implica tareas importantes

1. Asignación de los registros: que se refiere a elegir de todas las variables cuales deben ser asignadas a registros
2. Asignación a los registros: que se refiere a elegir en cual registro debe asignarse cada variable

4 principales categorías de registros:

- ax
- bx
- cx
- dx

Elección del orden de evaluación

Este aspecto se enfoca en elegir la secuencia correcta en la que deben ser ejecutadas las instrucciones, ya que una secuencia óptima acelerará la ejecución de los programas. Esto representa un problema complejo porque para elegir la mejor secuencia de ejecución, es necesario encontrar con un panorama más amplio en el contexto de ejecución, por ello inicialmente basta con hacer la traducción literal (línea a línea) de cada una de las instrucciones incluídas en la tabla de cuádruplos

Enfoque en la generación de código

Se refiere a que el código producido debe ser correcto y funcional por lo que durante la traducción debe asegurarse que sean incluídas las acciones de código necesarias para garantizar la correctitud del programa final